

## **ENEE459v 'S26 Exam01 : due no more than 4 hours after starting**

**Fill in the pledge**/comment below and paste it at the top of the main.c of your project submission.

```
/* Pledge: I, _____(your typed name)_____, pledge on my honor that I have not given or received any assistance from any other person on this examination */
```

**Issues with the exam:** Hawkins can be reached at 202-567-1319

**WHAT TO SUBMIT:** evaluation will be based on three components:

- plan of action (15%): brief (1-2 page) description written after reading the problem but before any programming, of how you plan to handle the problem's requirements. Be sure to mention subgoals and interim tests which you will perform during development.

- project (75%): the CubeIDE project which performs the 'dit-dah' display function.

**Note:** to allow evaluation your project must be based HW03starter\_ws.zip .

- test instructions (10%): information on how to demonstrate and observe your program's operation. Set aside time to write this even if your program isn't working yet.

### **The Gizmo: a Morse code trainer**

Big picture: the Gizmo is used to help people prepare for their commercial radiotelegraph operator's license exam by providing Morse code messages for them to decode.

The Gizmo will output a sequence of 800Hz square wave bursts ('dits' and 'dahs') to pin PA5 of the B-L475\_IOT1A representing the key-on tones of Morse code. When the sequence of tones is finished the user will type in its translation from Morse code as an ASCII string on the serial terminal and the program will confirm correct translation or mark errors. .

A data format for encoding Morse code messages has been developed for use by your program. Please read its description on the next page. You should use the test string "ABC " described there for your software development. A complete library of similar test strings is being developed in a different department for use when your program goes 'live'.

To start Gizmo operation the user enters a command of "D60" on the serial terminal, indicating they want to start a decoding session with a 'dot' duration of 60 milliseconds. Other durations from 20 to 300 are also reasonable.

### **WHAT TO SUBMIT :**

Submit on Canvas a single .zip file containing all three components (plan, project, test), as

<yourlastname>\_<thiscourse>\_<Exam01>\_<rev#>.zip (omit rev# if this is the first submit)  
(example: hawkins\_ENEE459v\_Exam01\_rev2.zip)

## Gizmo data description:

(Information on the underlying technology for Morse code is available in many places, but this [figure](#) from Wikipedia should tell you everything you need to know for this problem).

The Gizmo is a Morse code trainer implemented on the B-L475\_IOT1A microprocessor board. The first generation Gizmo operates in decode-training mode only. It generates sequences of ‘dots’ (also called ‘dits’) and ‘dashes’ (‘dahs’) representing Morse-encoded text strings which the user will try to decode.

The following convention is used to represent the ‘dits’ and ‘dahs’ of Morse code letters, strings, and sentences:

```
typedef struct ditdah
{
    int16_t ON; //positive values representing key-on (tone) duration
    int16_t OFF; //negative values representing key-off (silence) duration
} ditdah_t;
```

where the opposing signs for ON and OFF records act as markers to distinguish the type of action (key-on or key-off) to be taken.

Our example and the test pattern to use for your development was constructed by applying the rules of the [figure](#) in Wikipedia to a string “ABC ” (three letters with a trailing space) .

Character values taken from the Wikipedia figure are:

A: dot,dash

B: dash,dot,dot,dot

C: dash,dot,dash,dot

The string "ABC " (four characters when ‘space’ is included) expressed as a ditdah array at speed ‘t’:

```
ditdah_t ABCsp[] = {
    {+t,-t},{+3*t,(-t-2*t)}, // the letter ‘A’ is followed by another letter
    {+3*t,-t},{+t,-t},{+t,-t},{+t,(-t-2*t)}, // the letter ‘B’ is followed by another letter
    {+3*t,-t},{+t,-t},{+3*t,-t},{+t,(-t-6*t)} // the letter ‘C’ is followed by a space )
}
```

By taking the common factor ‘t’ out and reducing we create a normalized ditdah array

```
ditdah_t _ABCsp[] = {{1,-1},{3,-3},{3,-1},{1,-1},{1,-1},{1,-3},{3,-1},{1,-1},{3,-1},{1,-7}};
```

which can be used at any desired speed by scaling all values by the common factor ‘t’.

A commercial radiotelegraph license requires the operator to perform at 20 words per minute (wpm) which implies a value of  $t = 60$  msec when a standard set of words is tested.

The test/demonstration for your Gizmo should use the \_ABCsp ditdah array above with  $t = 60$ .

Reminder: the display on PA5 is a 800hz square wave ‘tone’ which is superimposed on the key-on part of the dits and dahs.